Authenticated Diffie-Hellman: Project Report

Logan Dalton
Computer Science Department
University of North Carolina Wilmington
Wilmington, North Carolina
Ltd6428@uncw.edu

1. Algorithms and examples worked:

Square-and-multiply- The modular_exponentiation function works through the exponent one bit at a time. Whenever the exponent is odd, the current base is multiplied into the result. After that, the base is squared and the exponent is cut in half. This continues until the exponent reaches zero. Because of this, the program uses a reduced amount of resources and is much more efficient, which keeps the math fast even when working with larger numbers.

Prime generation and validation- The generate_prime function only accepts a small, fixed range of bit sizes. It uses PyCryptodome's getPrime to create the prime value and then confirms that the bit length is correct and that the prime is odd. When Bob receives the prime value from Alice, he can't just trust it as it is. He has to ceck its type, its size, and runs the library's primality test on it, so a number that is not actually prime gets rejected.

Factoring and generators- Trial division is used to find each distinct prime factor of p minus one. A value is a generator only if raising it to p minus one divided by each factor never gives back one. If one factor returns one, then that value is thrown out and the next value is tested.

Key agreement- Alice and Bob each pick a random private value that stays in a safe locatuion. Each of them raises the generator to their private value to get a public value, and then they swap public values. Each person raises the other person's public value to their own private value, and both end up with the same shared secret without ever sending it.

Authentication- Alice and Bob receive each other's Ed25519 public keys through a trusted path. They exchange three messages, and the last two sign the complete message, including both nonces. Each side verifies the signatures with the key from its own trusted setup and rejects messages that are oversized/undersized, or mismatched. Any failure aborts the session and clears its secrets. Additionally, this design stops replays, since a replayed message can never finish without Alice signing Bob's new nonce.

2. Results:

Options Menu.

```
PS C:\Users\dalto\DH_Midterm> py main.py
1 = exchange, 2 = Eve's attack, 3 = benchmark:
```

Option 1.

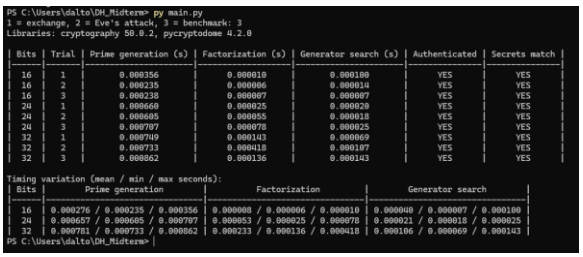
```
PS C:\Users\dalto\DH_Midterm> py main.py
1 = exchange, 2 = Eve's attack, 3 = benchmark: 1
Prime size in bits, 16 to 32 (press Enter for 16):
Requested prime size: 16 bits
Modulus: generated using library prime generator
Modulus bit length: 16
Received modulus library validation: PASS
Generator: validated
Alice initiation signature: VALID
Bob response signature: VALID
Alice finish signature: VALID
Transcript authenticated: YES
Shared-secret equality in local test: PASS
Explicit key confirmation: NOT IMPLEMENTED
PS C:\Users\dalto\DH_Midterm>
```

Option 2.

```
PS C:\Users\dalto\DH_Midterm> py main.py
1 = exchange, 2 = Eve's attack, 3 = benchmark: 2
Prime size in bits, 16 to 32 (press Enter for 16):
Attack against the unauthenticated version (p = 23, g = 5, a = 6, b = 15):
  e1 = 3, public value E1 = 5^3 mod 23 = 10
  e2 = 7, public value E2 = 5^7 mod 23 = 17
  Eve sends E1 to Alice as though it came from Bob, and E2 to Bob as though it came from Alice.
  Alice computes: 10^6 mod 23 = 6
  Eve computes:   8^3 mod 23 = 6
  Bob computes:   17^15 mod 23 = 15
  Eve computes:   19^7 mod 23 = 15
  Alice and Bob now hold different secrets, and Eve knows both.

Attack against the authenticated version (16-bit prime):
  Eve modified Bob's public value.
  Bob response signature: INVALID
  Exchange aborted.
```

Option 3.



Every exchange was authenticated, Alice and Bob always ended up with matching secrets. Without authentication, Eve sends her own public value to Alice and a different one to Bob. Alice and Bob end up with different secrets, and Eve knows both. With authentication, the same substitution breaks Bob's signature, and the exchange aborts. Prime generation took longer as the bit size grew. Factorization time varied the most at the largest size, because it depends on how p minus one happens to factor. Generator search stayed fast at every size.

3. Trust assumptions and limitations:

This design assumes an honest, trusted setup, private keys that stay private, and correct libraries. There are also several limitations. First, primes this small can be broken instantly. Second, there is no key confirmation, key derivation, or automated test suite. Third, the public value checks are basic and run in just one process.

4. Finite-field DH vs. ECDH:

Both protocols let two parties agree on a shared secret over a public domain. Finite-field DH relies on modular exponentiation. ECDH instead relies on scalar multiplication, which is the curve version of square-and-multiply and can be done with the Montgomery ladder [1].

Both depend on the discrete logarithm problem being hard. The primes in this project are insecure because the group is small enough to be searched directly. Furthermore, Pohlig-Hellman makes the search even faster when p minus one has small factors. NIST rates finite-field DH keys and much smaller elliptic curve keys at about the same strength [3]. This is why ECDH keys are so much smaller.

Factoring p minus one also limits this implementation. Verifying a generator requires every prime factor, and trial division works for small primes but is hopeless for the sizes used in real systems. Real systems avoid this problem by using curves where the group order is already known.

ECDH creates fresh keys for every exchange and discards them afterward, just as this project creates fresh private values each time. TLS 1.3 requires ephemeral key exchange and removed static RSA and static DH entirely [2]. This provides forward secrecy. If a long-term signing key leaks later, past session secrets still cannot be recovered.

However, ECDH still needs authentication. Without it, Eve's substitution attack works exactly as it does in this project. TLS 1.3 signs the transcript, which serves the same role as the Ed25519 signatures here [2]. Ed25519 and ECDH use related curves, but they serve different purposes and use separate keys.

Ultimately, production applications use established protocols, validated parameters, and vetted libraries, because details such as constant time code and parameter validation are easy to get wrong. The research found many servers using weak or widely shared DH groups that attackers could break with precomputation [4].

References

1. Langley, Hamburg, Turner, Elliptic Curves for Security, RFC 7748, 2016.

2. Rescorla, TLS Protocol Version 1.3, RFC 8446, 2018.

3. Barker, Recommendation for Key Management, NIST SP 800-57 Pt. 1 Rev. 5, 2020.

4. Adrian et al., Imperfect Forward Secrecy, ACM CC